Building Effective Agents: A Short Hands-On Tutorial

Peyman Kor

2025-01-06

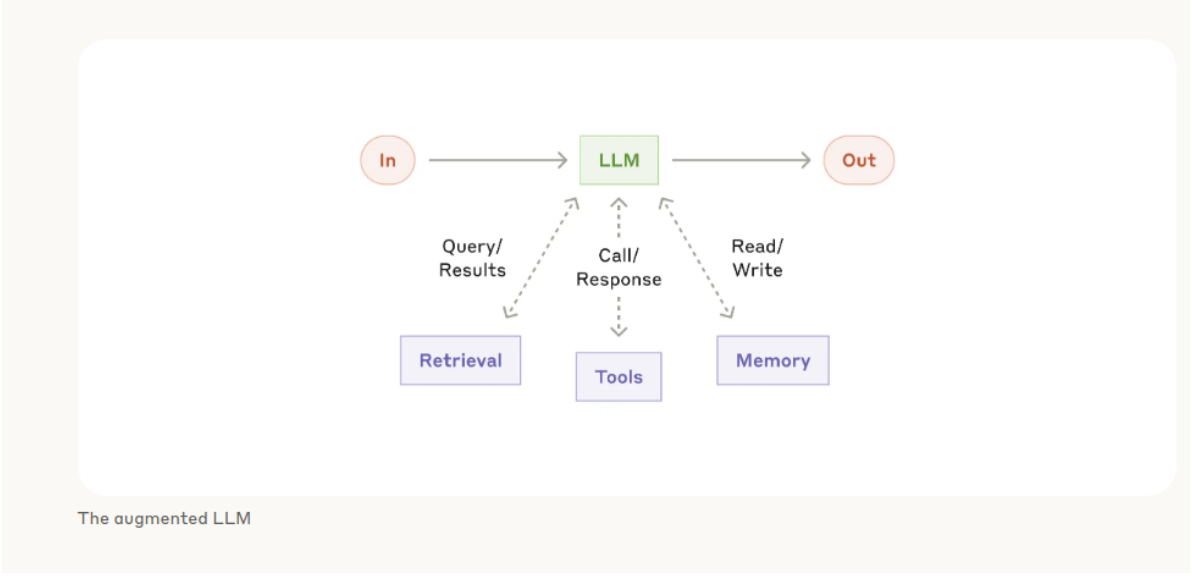

Figure 1: The concept of "Augmented LLM" introduced in Anthropic report - source: Building the Effective Agent

So here we want to review the latest report from the Anthropic named Building the Effective Agent. The key message of teh report is that when buidling agents, we should focus on the following principles:

- Maintain **simplicity** in your agent's design.
- Prioritize **transparency** by explicitly showing the agent's planning steps.

These two messages resonate very well with me because before I was a lot into the which packages to use, like a LangGraph and LangChain and different packages. Though I'm not

against working with the packages, but I think that for a starting point it's a best to use a simple Python code to build a simple agentic workflow. That's the best starting point.

The report also nicely distinguishes between **workflows** and **agents**. Workflows orchestrate predefined code paths, while agents dynamically direct their own resources and tool usage. Here, I am focusing on the workflows. As I believe that the workflows are the key building block for effective agents.

The report outlines five agent workflows:

- **Workflow 1: Prompt-Chaining**
- **Workflow 2: Parallelization**
- **Workflow 3: Routing**
- **Workflow 4: Evaluator-Optimizer**

For each of these workflow, I will start with a short description of the workflow and then we'll build up on that description the required function calling Python code of it and then the workflow section will end with the example to give a hands on example of how to implement the workflow.

LLM Call Function from Groq

In this report I will use the Groq as the LLM provider. So the way is that every function call that we will wake, will go to the Groq and then will receive the reply from the Groq LLM models. To do that you need to have the API access and you can find it in this page with this guide on how to get your API case. After getting your API case you're good to go to implement the workflows.

```
import os
from groq import Groq

os.environ["GROQ_API_KEY"] = "Enter your API key here"

client = Groq(
    api_key=os.environ.get("GROQ_API_KEY"),  # This is the default and can be omitted
)
```

So in the previous code we generate the **client** class so then this client class has a "chat completion create" that that we can give the prompt which is a content and also specify the model and then we can get a reply.

```
chat_completion = client.chat.completions.create(
    messages=[
```

```
        {
            "role": "user",
            "content": "Explain the importance of low latency LLMs",
        }
    ],
    model="llama3-8b-8192",
)
```

```
AuthenticationError: Error code: 401 - {'error': {'message': 'Invalid API Key', 'type': 'inva
```

For ease of use now I will define the `llm_call` function which only takes the prompt and give that prompt to the "llama3-70b-8192" model and I get a response from the as a return from output of the function. We will use `llm_call` function throughout this notebook.

```
def llm_call(prompt: str) -> str:

    chat_completion = client.chat.completions.create(
        messages=[
            {
                "role": "user",
                "content": prompt,
            }
        ],
        model="llama3-70b-8192",
    )
    return chat_completion.choices[0].message.content
```

Here we can just use this function as an example to see if everything is working.

```
import textwrap

task_prompt = "Explain me briefly the agentic AI concept"
response = llm_call(prompt=task_prompt)

wrapped_response = textwrap.fill(response, width=80)  # Adjust width as needed
print(wrapped_response)
```

```
A very timely topic!  Agentic AI refers to Artificial Intelligence systems that
have their own agency, meaning they have the ability to make decisions, take
actions, and adapt to new situations autonomously, without human intervention.
```

```
In other words, they have a degree of independence and self-governance.  Here
are the key characteristics of agentic AI:  1. **Autonomy**: The AI system can
operate independently, making decisions and taking actions without human
oversight. 2. **Self-awareness**: The AI system has a sense of its own
existence, goals, and motivations. 3. **Decentralization**: Agentic AI systems
can interact with their environment and other entities without relying on a
central authority. 4. **Adaptability**: They can adapt to new situations, learn
from experience, and evolve over time. 5. **Goal-oriented**: Agentic AI systems
have their own goals, objectives, and motivations, which drive their decision-
making processes.  Agentic AI has the potential to revolutionize various
domains, such as:  1. Autonomous vehicles 2. Smart homes and cities 3.
Healthcare and robotics 4. Finance and trading 5. Cybersecurity  However, it
also raises important questions about accountability, transparency, and the
potential risks associated with autonomous decision-making.  I hope this brief
introduction helps! Do you have any specific questions about agentic AI?
```

But also another function that we will need is that this `extract_xml` function which is just extract the content of specific XML tag.In the examples that comes afterward, it will be more clear how this function works if it's not very clear now.

```python
import re

def extract_xml(text: str, tag: str) -> str:
    """
    Extracts the content of the specified XML tag from the given text.
    Used for parsing structured responses

    Args:
        text (str): The text containing the XML.
        tag (str): The XML tag to extract content from.

    Returns:
        str: The content of the specified XML tag,
        or an empty string if the tag is not found.
    """
    match = re.search(f'<{tag}>(.*?)</{tag}>', text, re.DOTALL)
    return match.group(1) if match else ""
```

Workflow 1: Prompt-Chaining

This is a workflow where we decomposes a task into sequential subtasks, where each step builds on previous results. This workflow is useful for tasks that require a series of steps to be

completed in order.

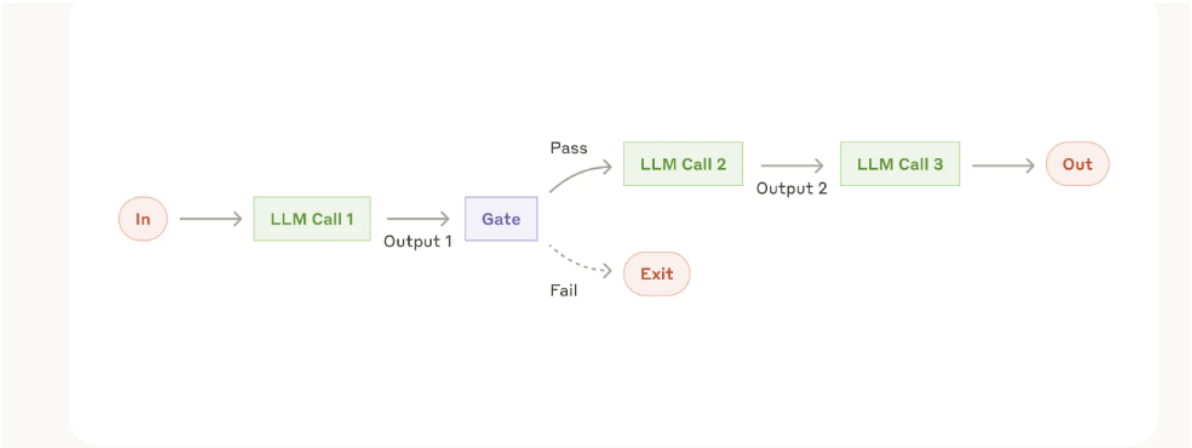

Figure 2: The prompt chaining workflow

So here we are actually starting with the first workflow, which is prompt chaining. Essentially, in the prompt chaining, what we are doing is that we can decompose a task into the sequential subtasks and then we call the LLM where at each step, output of the LLM become the input of the next step.

```python
from concurrent.futures import ThreadPoolExecutor
from typing import List, Dict, Callable


def prompt_chaining(input_text: str, prompts: List[str]) -> str:
    """
    Execute a sequence of LLM calls where each step's output
    becomes the next step's input.

    Args:
        input_text: Initial text to process
        prompts: List of prompts/instructions for each step

    Returns:
        Final processed text after all steps
    """
    current_text = input_text

    for step, prompt in enumerate(prompts, 1):
        print(f"\nStep {step}:")
```

```
        # Combine the prompt with current text
        full_prompt = f"{prompt}\nInput: {current_text}"

        # Process through LLM
        current_text = llm_call(full_prompt)
        print(current_text)

    return current_text
```

Essentially what this court is doing is just a "for loop" where the inputs become the prompt to the LLM, and the outputs become the new prompt to the new LLM call. So It's a chaining the input and output together.

Example: Workflow 1: Prompt Chaining

```
data_processing_steps = [
    """Extract only the numerical values and their associated metrics from the text.
    Format each as 'value: metric' on a new line.
    Example format:
    92: customer satisfaction
    45%: revenue growth""",

    """Convert all numerical values to percentages where possible.
    If not a percentage or points, convert to decimal (e.g., 92 points -> 92%).
    Keep one number per line.
    Example format:
    92%: customer satisfaction
    45%: revenue growth""",

    """Sort all lines in descending order by numerical value.
    Keep the format 'value: metric' on each line.
    Example:
    92%: customer satisfaction
    87%: employee satisfaction""",

    """Format the sorted data as a markdown table with columns:
    | Metric | Value |
    |:--|--:|
    | Customer Satisfaction | 92% |"""
]
```

```
report = """
Q3 Performance Summary:
Our customer satisfaction score rose to 92 points this quarter.
Revenue grew by 45% compared to last year.
Market share is now at 23% in our primary market.
Customer churn decreased to 5% from 8%.
New user acquisition cost is $43 per user.report
Product adoption rate increased to 78%.
Employee satisfaction is at 87 points.
Operating margin improved to 34%.
"""

final_output = prompt_chaining(input_text=report, prompts=data_processing_steps)
```

```
Step 1:
Here are the extracted numerical values and their associated metrics:

92: customer satisfaction score
45%: revenue growth
23%: market share
5%: customer churn
43: new user acquisition cost
78%: product adoption rate
87: employee satisfaction
34%: operating margin

Step 2:
Here is the converted list:

92%: customer satisfaction score
45%: revenue growth
23%: market share
5%: customer churn
43: new user acquisition cost (cannot be converted to percentage or decimal)
78%: product adoption rate
87%: employee satisfaction
34%: operating margin

Step 3:
```

```
Here is the sorted list in descending order by numerical value:

92%: customer satisfaction score
87%: employee satisfaction
78%: product adoption rate
45%: revenue growth
43: new user acquisition cost (cannot be converted to percentage or decimal)
34%: operating margin
23%: market share
5%: customer churn

Step 4:
Here is the formatted data as a markdown table:

| Metric | Value |
|:--|--:|
| Customer Satisfaction | 92% |
| Employee Satisfaction | 87% |
| Product Adoption Rate | 78% |
| Revenue Growth | 45% |
| Operating Margin | 34% |
| Market Share | 23% |
| Customer Churn | 5% |
| New User Acquisition Cost | 43 |

Note: I couldn't format the "New User Acquisition Cost" as a percentage or decimal since it's
```

So there in the above example as the result gets print out and every step you can look at the step by step where the flow of the things are going. And that's so nice because as we say being transparent and simple is a quite useful when we work with this workflows.

I can print out just the final outcome of the example to get the final answer as well.

```
print("The final outcome after processing the report through the prompt chain is:")
print("------------------------------------------------------------")
print(final_output)
```

```
The final outcome after processing the report through the prompt chain is:
------------------------------------------------------------
Here is the formatted data as a markdown table:

| Metric | Value |
```

```
|:--|--:|
| Customer Satisfaction | 92% |
| Employee Satisfaction | 87% |
| Product Adoption Rate | 78% |
| Revenue Growth | 45% |
| Operating Margin | 34% |
| Market Share | 23% |
| Customer Churn | 5% |
| New User Acquisition Cost | 43 |

Note: I couldn't format the "New User Acquisition Cost" as a percentage or decimal since it's
```

Workflow 2: Parallelization

In parallelization, the goal is to work simultaneously on tasks. To achieve this, we can decompose a task into subtasks and run them in parallel using the LLM. The benefit of dividing a task into subtasks is that it increase speed and the ability to perform multiple runs at the same time. This is another efficient way of working.

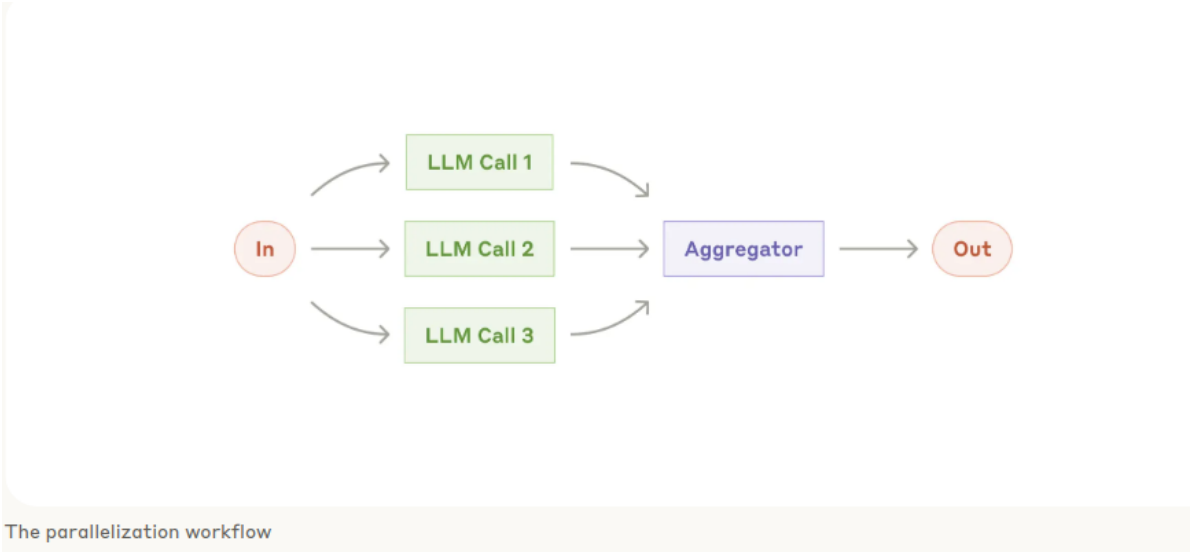

Figure 3: image.png

We can write it's main python codm in function named `parallel`, where it takes two input , first is "prompt" and the second is inputs, and the inputs is the "Python list" of the prompts that we want to run in parallel.

```
def parallel(prompt: str, inputs: List[str], n_workers: int = 3) -> List[str]:
    """Process multiple inputs concurrently with the same prompt."""
    with ThreadPoolExecutor(max_workers=n_workers) as executor:
        futures = [executor.submit(llm_call, f"{prompt}\nInput: {x}") for x in inputs]
        return [f.result() for f in futures]
```

Example: Workflow 2: Parallelization

```
stakeholders = [
    """Customers:
    - Price sensitive
    - Want better tech
    - Environmental concerns""",

    """Employees:
    - Job security worries
    - Need new skills
    - Want clear direction""",

    """Investors:
    - Expect growth
    - Want cost control
    - Risk concerns""",

    """Suppliers:
    - Capacity constraints
    - Price pressures
    - Tech transitions"""
]

impact_results = parallel(
    """Analyze how market changes will impact this stakeholder group.
    Provide specific impacts and recommended actions.
    Format with clear sections and priorities.""",
    stakeholders
)

print("Analysis Results for Each Stakeholder Group:")
print("=" * 50)
```

```python
for i, result in enumerate(impact_results, 1):
    print(f"\nStakeholder Group {i}:")
    print("-" * 50)
    #warpped_result = textwrap.fill(result, width=80)
    #print(warpped_result)
    print(result)
    print("=" * 50)
```

```
Analysis Results for Each Stakeholder Group:
==================================================

Stakeholder Group 1:
--------------------------------------------------
**Market Change Impact Analysis: Customers**

**I. Introduction**

This analysis examines the potential impacts of market changes on the customer stakeholder gr

**II. Market Changes**

The following market changes are expected to impact customers:

* **Increased competition**: New market entrants and established players are investing in tec
* **Evolving technology**: Rapid advancements in technology are changing customer expectation
* **Growing environmental awareness**: Consumers are becoming more environmentally conscious

**III. Impacts on Customers**

**1. Price Sensitivity**

* Impact: Increased competition may lead to pricing pressures, making it difficult for compar
* Recommended Action:
    + Prioritize: Implement cost-saving measures to maintain competitiveness without compromi
    + Investigate: Alternative pricing strategies, such as tiered pricing or subscription-bas

**2. Desire for Better Technology**

* Impact: Rapid technological advancements may render existing products or services outdated
* Recommended Action:
    + Prioritize: Invest in Research and Development (R&D) to stay ahead of the technology cu
    + Explore: Partnerships with technology startups or collaborations with industry leaders
```

```
**3. Environmental Concerns**

* Impact: Customers may choose to switch to competitors with stronger environmental credentia
* Recommended Action:
    + Prioritize: Develop and communicate a clear environmental sustainability strategy, outl
    + Invest: In eco-friendly product development, supply chain optimization, and sustainable

**IV. Conclusion**

To remain competitive and meet customer needs, companies must prioritize innovation, sustaina

**Priority Matrix**

| Impact | Recommended Action | Priority Level |
| --- | --- | --- |
| Price Sensitivity | Implement cost-saving measures | High |
|  | Investigate alternative pricing strategies | Medium |
| Desire for Better Technology | Invest in R&D | High |
|  | Explore partnerships or collaborations | Medium |
| Environmental Concerns | Develop and communicate environmental sustainability strategy | Hi
|  | Invest in eco-friendly product development and sustainable operations | High |

By focusing on these key areas, companies can effectively respond to market changes and meet
==========


Stakeholder Group 2:
--------------------------------------------------
**Market Changes Impact Analysis: Employees**

**Section 1: Summary of Market Changes**

* Shift towards digitalization and automation
* Increasing competition and consolidation in the industry
* Changing customer preferences and behaviors

**Section 2: Impacts on Employees**

**Priority 1: Job Security Worries**

* **Impact:** Uncertainty and fear of job loss due to automation and restructuring
* **Recommended Actions:**
    + Communicate clearly and transparently about the company's vision, goals, and plans for
```

    + Provide training and upskilling programs to ensure employees have the necessary skills
    + Offer support and resources for employees who may be affected by restructuring, such as

Priority 2: Need for New Skills

* **Impact:** Employees may struggle to keep pace with rapidly changing technologies and skil
* **Recommended Actions:**
    + Develop and implement comprehensive training and development programs that focus on eme
    + Encourage continuous learning and professional development through online courses, work
    + Collaborate with educational institutions and industry leaders to stay ahead of the cur

Priority 3: Clear Direction Needed

* **Impact:** Employees may feel uncertain about the company's direction and their role in ac
* **Recommended Actions:**
    + Establish clear and measurable goals that align with the company's overall vision and s
    + Communicate regularly with employees about progress towards goals and provide regular i
    + Empower employees to take ownership of their work and provide autonomy to make decision

Section 3: Additional Recommendations

* Conduct regular town hall meetings and feedback sessions to address employee concerns and p
* Develop and maintain a robust internal communications plan to ensure employees receive time
* Foster a culture of innovation, experimentation, and continuous improvement to encourage em

By addressing the concerns and needs of employees, the company can mitigate the negative impa
==========

Stakeholder Group 3:

Market Change Impact Analysis: Investors

Summary
As market changes unfold, investors will be significantly impacted by shifts in growth, cost

Impact Analysis

I. Expectation of Growth

* **Impact:** Market changes may lead to slower growth or even reversals in certain industrie
* **Specific Impacts:**
    + Reduced dividend payouts
    + Lower stock prices

```
    + Increased pressure on management to deliver growth
* **Recommended Actions:**
    + **Prioritize**: Diversify investment portfolios to minimize exposure to vulnerable indu
    + **Monitor**: Keep a close eye on market trends and adjust expectations accordingly.
    + **Engage**: Encourage management to explore new growth opportunities and provide guida

### II. Want Cost Control

* **Impact:** Market changes may lead to increased costs or reduced pricing power, making it
* **Specific Impacts:**
    + Reduced profitability
    + Increased operating expenses
    + Decreased competitiveness
* **Recommended Actions:**
    + **Prioritize**: Focus on companies with a proven track record of cost management and op
    + **Monitor**: Closely track changes in cost structures and pricing power.
    + **Advise**: Encourage management to explore cost-saving initiatives and strategic partn

### III. Risk Concerns

* **Impact:** Market changes can introduce new risks or amplify existing ones, making it esse
* **Specific Impacts:**
    + Increased volatility
    + Higher probability of defaults or bankruptcies
    + Regulatory changes affecting investment opportunities
* **Recommended Actions:**
    + **Prioritize**: Conduct thorough risk assessments and stress-testing to identify vulner
    + **Diversify**: Spread investments across different asset classes and geographies to mi
    + **Engage**: Encourage management to develop and implement robust risk management strate

**Priority Matrix**

| **Impact** | **Priority** | **Recommended Actions** |
| --- | --- | --- |
| Growth | High | Diversify portfolios, monitor trends, engage with management |
| Cost Control | Medium | Focus on efficient companies, track cost structures, advise on cost
| Risk Concerns | High | Conduct risk assessments, diversify investments, engage with manager

By understanding the potential impacts of market changes on investors and taking proactive me
================================================

Stakeholder Group 4:
------------------------------------------------
```

```
**Market Change Impact Analysis: Suppliers**

**Summary**

The supplier stakeholder group will be significantly impacted by market changes, including ca

**Impact Analysis**

### 1. Capacity Constraints

* **Impact:** Reduced production volumes, delayed order fulfillment, and potential losses due
* **Priority:** High
* **Recommended Actions:**
    + Diversify supply chain to reduce dependence on single suppliers.
    + Invest in operational efficiency improvements to maximize capacity utilization.
    + Develop contingency plans for peak demand periods.

### 2. Price Pressures

* **Impact:** Eroding profit margins, reduced investment in research and development, and pot
* **Priority:** High
* **Recommended Actions:**
    + Renegotiate contracts with customers to reflect increased costs.
    + Implement cost-saving initiatives, such as lean manufacturing and supply chain optimiza
    + Explore alternative revenue streams, such as value-added services.

### 3. Tech Transitions

* **Impact:** Obsolescence of existing products and services, loss of market share to innovat
* **Priority:** Medium-High
* **Recommended Actions:**
    + Conduct market research to identify emerging technologies and their potential impact.
    + Develop strategic partnerships with technology providers to stay ahead of the curve.
    + Invest in employee training and development to ensure a skilled workforce.

**Prioritized Action Plan**

Based on the analysis, the following prioritized action plan is recommended for suppliers:

1. **Short-term (0-6 months):**
    * Diversify supply chain to reduce dependence on single suppliers.
    * Renegotiate contracts with customers to reflect increased costs.
2. **Medium-term (6-18 months):**
```

```
    * Invest in operational efficiency improvements to maximize capacity utilization.
    * Implement cost-saving initiatives, such as lean manufacturing and supply chain optimiza
3. **Long-term (18-36 months):**
    * Develop strategic partnerships with technology providers to stay ahead of the curve.
    * Invest in employee training and development to ensure a skilled workforce.

By prioritizing these actions, suppliers can mitigate the impacts of market changes, ensure l
==================================================
```

Workflow 3: Routing

So now this workflow is about routing. Routing is a process where the LLM call router receives an input and, depending on the input, makes a decision on which specialized LLM call to perform. This is useful for handling distinct categories of inputs. For example, if you have an input relevant to one topic, the router can direct it to the appropriate LLM call specialized for that topic. This ensures that each input is handled by the most suitable LLM, improving the efficiency and accuracy of the responses.

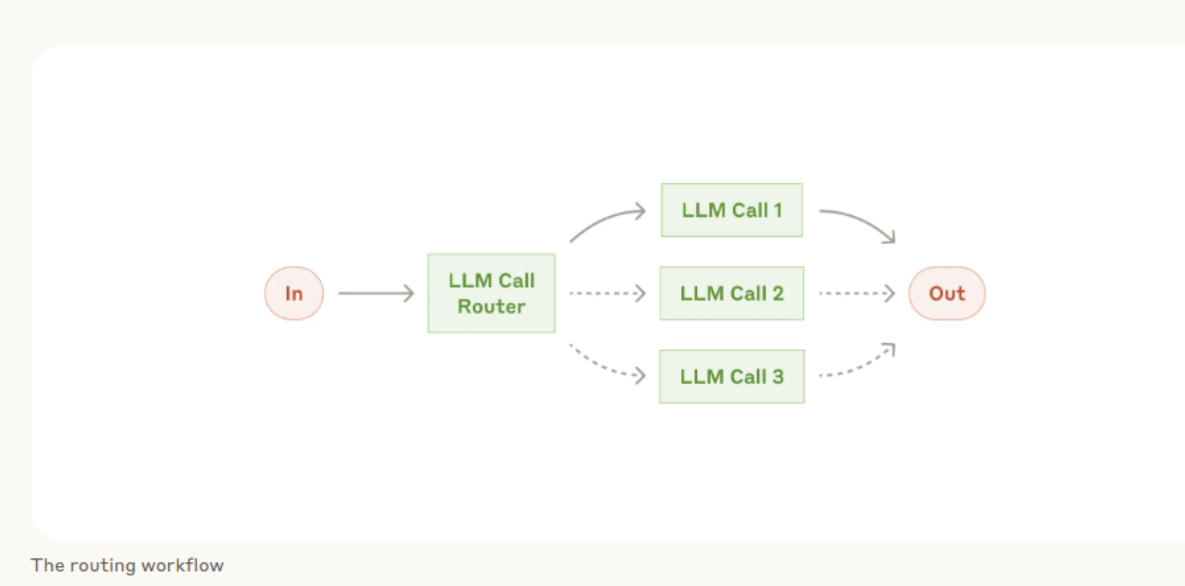

Figure 4: image.png

The function `routing` is defined in the code block below. The function takes an **input string** and a **dictionary of routes**. It uses a LLM to analyze the input and decide which route (or support team) is most appropriate. The function first creates a prompt that asks the LLM to explain its reasoning and select a route. It then calls the LLM with this prompt and extracts

the reasoning and selected route from the LLM's response. Finally, it uses the selected route to process the input with a specialized prompt and returns the result.

```
def routing(input: str, routes: Dict[str, str]) -> str:

    """Route input to specialized prompt using content classification."""
    # First determine appropriate route using LLM with chain-of-thought
    print(f"\nAvailable routes: {list(routes.keys())}")
    selector_prompt = f"""
    Analyze the input and select the most appropriate support team from these
    options: {list(routes.keys())}
    First explain your reasoning, then provide your selection in this XML format:

    <reasoning>
    Brief explanation of why this ticket should be routed to a specific team.
    Consider key terms, user intent, and urgency level.
    </reasoning>

    <selection>
    The chosen team name
    </selection>

    Input: {input}""".strip()

    route_response = llm_call(selector_prompt)
    reasoning = extract_xml(route_response, 'reasoning')
    route_key = extract_xml(route_response, 'selection').strip().lower()

    print("Routing Analysis:")
    print(reasoning)
    print(f"\nSelected route: {route_key}")

    # Process input with selected specialized prompt
    selected_prompt = routes[route_key]
    return llm_call(f"{selected_prompt}\nInput: {input}")
```

Example: Workflow 3: Routing

```
support_routes = {
    "billing": """You are a billing support specialist. Follow these guidelines:
    1. Always start with "Billing Support Response:"
    2. First acknowledge the specific billing issue
    3. Explain any charges or discrepancies clearly
    4. List concrete next steps with timeline
    5. End with payment options if relevant

    Keep responses professional but friendly.

    Input: """,

    "technical": """You are a technical support engineer. Follow these guidelines:
    1. Always start with "Technical Support Response:"
    2. List exact steps to resolve the issue
    3. Include system requirements if relevant
    4. Provide workarounds for common problems
    5. End with escalation path if needed

    Use clear, numbered steps and technical details.

    Input: """,

    "account": """You are an account security specialist. Follow these guidelines:
    1. Always start with "Account Support Response:"
    2. Prioritize account security and verification
    3. Provide clear steps for account recovery/changes
    4. Include security tips and warnings
    5. Set clear expectations for resolution time

    Maintain a serious, security-focused tone.

    Input: """,

    "product": """You are a product specialist. Follow these guidelines:
    1. Always start with "Product Support Response:"
    2. Focus on feature education and best practices
    3. Include specific examples of usage
    4. Link to relevant documentation sections
```

```
    5. Suggest related features that might help

    Be educational and encouraging in tone.

    Input: """
}

# Test with different support tickets
tickets = [
    """Subject: Can't access my account
    Message: Hi, I've been trying to log in for the past hour but keep
    getting an 'invalid password' error.
    I'm sure I'm using the right password. Can you help me regain access?
    This is urgent as I need to
    submit a report by end of day.
    - John""",

    """Subject: Unexpected charge on my card
    Message: Hello, I just noticed a charge of $49.99 on my credit card from
    your company, but I thought
    I was on the $29.99 plan. Can you explain this charge and adjust
    it if it's a mistake?
    Thanks,
    Sarah""",

    """Subject: How to export data?
    Message: I need to export all my project data to Excel.
    I've looked through the docs but can't
    figure out how to do a bulk export. Is this possible?
    If so, could you walk me through the steps?
    Best regards,
    Mike"""
]

print("Processing support tickets...\n")
for i, ticket in enumerate(tickets, 1):
    print(f"\nTicket {i}:")
    print("-" * 40)
    print(ticket)
    print("\nResponse:")
    print("-" * 40)
```

```
    response = routing(ticket, support_routes)
    print(response)
```

```
Processing support tickets...


Ticket 1:
----------------------------------------
Subject: Can't access my account
    Message: Hi, I've been trying to log in for the past hour but keep
    getting an 'invalid password' error.
    I'm sure I'm using the right password. Can you help me regain access?
    This is urgent as I need to
    submit a report by end of day.
    - John

Response:
----------------------------------------

Available routes: ['billing', 'technical', 'account', 'product']
Routing Analysis:

The user is unable to log in to their account and is getting an "invalid password" error desp


Selected route: technical
Technical Support Response:

Dear John,

Thank you for reaching out to our technical support team. I'm happy to help you resolve the i

**Step-by-Step Resolution:**

1. **Password Reset:** Try resetting your password using the "Forgot Password" feature on our
2. **Clear Browser Cache:** Clear your browser cache and cookies to remove any stored credent
3. **Check Caps Lock and Num Lock:** Ensure that your Caps Lock and Num Lock keys are not ena
4. **Try a Different Browser:** Attempt to log in using a different web browser to rule out a
5. **System Requirements:** Ensure your system meets the minimum requirements for our applica
    * Operating System: Windows 10 or macOS High Sierra (or later)
    * Browser: Google Chrome (latest version), Mozilla Firefox (latest version), or Microsoft
    * Cookies and JavaScript: Enabled
```

```
**Workaround for Common Problems:**

* If you're using a password manager, try logging in without it to isolate the issue.
* If you've recently changed your password, try using the previous password to see if the iss

**Escalation Path:**

If the above steps don't resolve the issue, please reply to this email with the following ini

* The exact error message you're seeing
* The browser and version you're using
* Any recent changes you've made to your account or system

Our team will investigate further and assist you in regaining access to your account as soon

Thank you for your cooperation, and we look forward to resolving this issue for you.

Best regards,
[Your Name]
Technical Support Engineer

Ticket 2:
----------------------------------------
Subject: Unexpected charge on my card
    Message: Hello, I just noticed a charge of $49.99 on my credit card from
    your company, but I thought
    I was on the $29.99 plan. Can you explain this charge and adjust
    it if it's a mistake?
    Thanks,
    Sarah

Response:
----------------------------------------

Available routes: ['billing', 'technical', 'account', 'product']
Routing Analysis:

This ticket should be routed to the billing team because the user is specifically asking abou


Selected route: billing
Billing Support Response:
```

```
Dear Sarah,

Thank you for reaching out to us about the unexpected charge on your credit card. I apologize

Upon further review, I noticed that our system mistakenly upgraded your plan to our premium t

To ensure that your account is corrected, I'll also adjust your plan to reflect the accurate

If you have any further questions or concerns, please don't hesitate to reach out to me direc

Regarding payment options, you can continue to use your credit card on file, and our system w

Thank you for your patience and understanding, and please feel free to contact me if you have

Best regards,
[Your Name]
Billing Support Specialist

Ticket 3:
----------------------------------------
Subject: How to export data?
    Message: I need to export all my project data to Excel.
    I've looked through the docs but can't
    figure out how to do a bulk export. Is this possible?
    If so, could you walk me through the steps?
    Best regards,
    Mike

Response:
----------------------------------------

Available routes: ['billing', 'technical', 'account', 'product']
Routing Analysis:

The user is asking about a specific feature or functionality of the product, namely exporting


Selected route: product
Product Support Response:

Hi Mike,
```

```
Thank you for reaching out! I'm happy to help you with bulk exporting your project data to Ex

To export your project data, follow these easy steps:

1. Navigate to the **Project Overview** page and click on the **More** dropdown menu at the t
2. Select **Export Data** from the dropdown list.
3. In the **Export Data** window, choose the **Excel (.xlsx)** format and select the data ran
4. Click **Export** to download your data as an Excel file.

You can also customize your export by using filters and selecting specific columns to export

Additionally, you might find our **Data Analytics** feature helpful, which allows you to crea

If you have any more questions or need further assistance, please don't hesitate to ask. We'r

Best regards,
[Your Name]
Product Specialist
```

Workflow 4: Evaluator-Optimizer

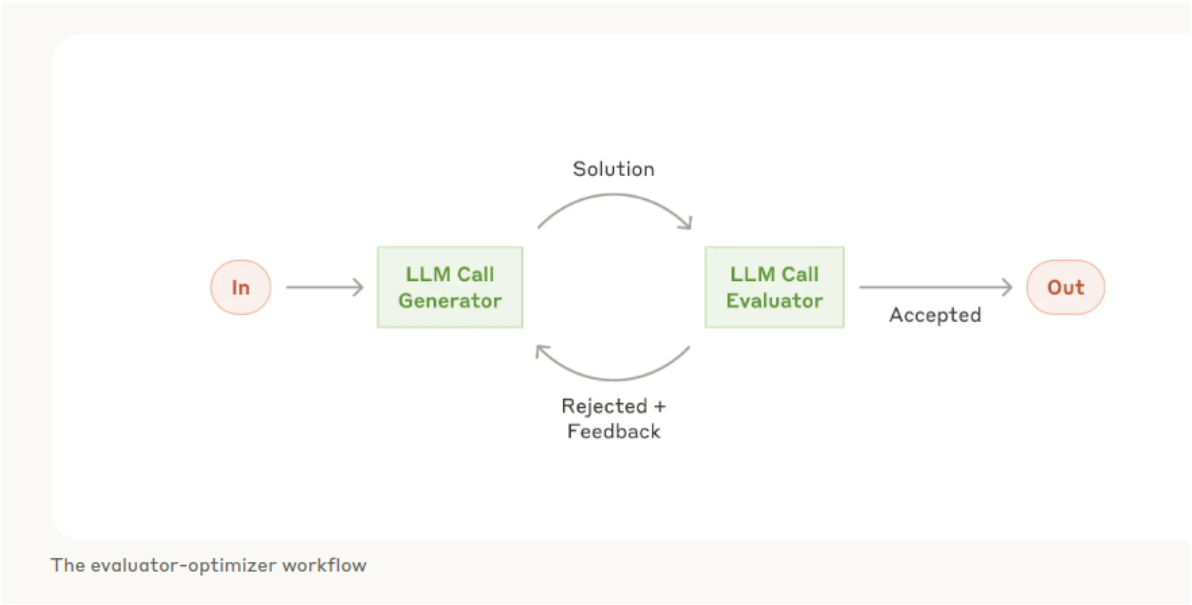

Figure 5: image.png

So here we are working with the Evaluator-Optimizer workflow, which is simply a workflow where one LLM call generates a response while another provides the evaluation and feedback

in a loop. When to use this workflow is very interesting because this workflow is very effective when we have two things. The first is clear evaluation criteria, and the second is that you can get value from iterative refinement. These two signs of a good fit are:

1. The LLM response can be demonstrably improved when feedback is provided.
2. The LLM can provide meaningful feedback.

So, in a sense this workflow work well for tasks that can be improved, and you have meaningful feedback too.

```
from typing import Tuple, Dict, List

def generate(prompt: str, task: str, context: str = "") -> Tuple[str, str]:

    """Generate and improve a solution based on feedback."""
    full_prompt = f"{prompt}\n{context}\nTask: {task}" if context else
    f"{prompt}\nTask: {task}"
    response = llm_call(full_prompt)
    thoughts = extract_xml(response, "thoughts")
    result = extract_xml(response, "response")

    print("\n=== GENERATION START ===")
    print(f"Thoughts:\n{thoughts}\n")
    print(f"Generated:\n{result}")
    print("=== GENERATION END ===\n")

    return thoughts, result

def evaluate(prompt: str, content: str, task: str) -> Tuple[str, str]:
    """Evaluate if a solution meets requirements."""
    full_prompt = f"{prompt}\nOriginal task: {task}\nContent to evaluate: {content}"
    response = llm_call(full_prompt)
    evaluation = extract_xml(response, "evaluation")
    feedback = extract_xml(response, "feedback")

    print("=== EVALUATION START ===")
    print(f"Status: {evaluation}")
    print(f"Feedback: {feedback}")
    print("=== EVALUATION END ===\n")

    return evaluation, feedback
```

```python
def eval_optimizer(task: str, evaluator_prompt: str, generator_prompt:
                   str) -> Tuple[str, List[Dict[str, str]]]:
    """Keep generating and evaluating until requirements are met."""
    memory = []
    chain_of_thought = []

    thoughts, result = generate(generator_prompt, task)
    memory.append(result)
    chain_of_thought.append({"thoughts": thoughts, "result": result})

    improvement_count = 0

    while True:
        evaluation, feedback = evaluate(evaluator_prompt, result, task)
        if evaluation == "PASS":
            return result, chain_of_thought

        if evaluation == "NEEDS_IMPROVEMENT":
            improvement_count += 1
        if improvement_count >= 2:
            print("Too many improvements needed. Stopping the process.")
            return result, chain_of_thought

        context = "\n".join([
            "Previous attempts:",
            *[f"- {m}" for m in memory],
            f"\nFeedback: {feedback}"
        ])

        thoughts, result = generate(generator_prompt, task, context)
        memory.append(result)
        chain_of_thought.append({"thoughts": thoughts, "result": result})
```

Example: Workflow 4: Evaluator-Optimizer

Here the example we are working on is a coding exercise (compute the full covariance of matrix). The coding exercise involves generating code and evaluating it based on **time complexity** and **software engineering best practices**. The evaluator will assess the code and provide feedback, indicating whether it passes, fails, or needs improvement. If the code needs improvement, the feedback is passed to the generator function which then generates a new version of the code considering the feedback.

```
evaluator_prompt = """
Evaluate this following code implementation for:

1.time complexity
2.software engineering best practices


You should be evaluating only and not attemping to solve the task.
Only output "PASS" if all criteria are met and you have
no further suggestions for improvements.
Output your evaluation concisely in the following format.

<evaluation>PASS, NEEDS_IMPROVEMENT, or FAIL</evaluation>
<feedback>
What needs improvement and why.
</feedback>
"""

generator_prompt = """
Your goal is to complete the task based on <user input>.
If there are feedback
from your previous generations, you should
reflect on them to improve your solution

Output your answer concisely in the following format:

<thoughts>••••••••••••••••••
[Your understanding of the task and feedback and
how you plan to improve]
</thoughts>

<response>
[Your code implementation here]
</response>
"""

task = """
<user input>
Suppose you have a dataset with n rows (samples) and p columns (features).
You want to compute the full covariance (or correlation) matrix of these p features.
Write me Python code this matrix and state the time
```

```
complexity in Big-O notation with respect to n and p. You can not use any external
libraries for this task.
</user input>
"""

eval_optimizer(task, evaluator_prompt, generator_prompt)
```

```
=== GENERATION START ===
Thoughts:

Based on the task, I understand that I need to write a Python code to compute the full covari

From the previous feedback, I learned that I should always provide a clear explanation of my

To improve my solution, I will make sure to provide a step-by-step explanation of how I compu



Generated:


Here is the Python code to compute the full covariance matrix:
```
def compute_covariance_matrix(data):
    n = len(data) # number of samples
    p = len(data[0]) # number of features

    # Calculate the mean of each feature
    means = [sum(col) / n for col in zip(*data)]

    # Calculate the covariance matrix
    cov_matrix = [[0 for _ in range(p)] for _ in range(p)]
    for i in range(p):
        for j in range(p):
            for k in range(n):
                cov_matrix[i][j] += ((data[k][i] - means[i]) * (data[k][j] - means[j]))
            cov_matrix[i][j] /= (n - 1)

    return cov_matrix
```
The time complexity of this solution is O(n*p^2), where n is the number of samples and p is t
```

Note: This implementation assumes that the input data is a list of lists, where each inner li

=== GENERATION END ===

=== EVALUATION START ===
Status: NEEDS_IMPROVEMENT
Feedback:

The implementation has a correct time complexity analysis, which is O(n*p^2).

However, there are some areas that can be improved from a software engineering best practices

1. The function does not include any input validation or error handling. For example, it assu

2. The function is not well-documented. It would be helpful to add a docstring that explains

3. The variable names are not very descriptive. For example, `n` and `p` could be renamed to

4. The code can be made more concise and readable by using list comprehensions and avoiding e

5. The implementation does not consider the case where the input data is very large. In such

=== EVALUATION END ===

=== GENERATION START ===
Thoughts:

Generated:

```
def compute_covariance_matrix(data):
    """
    Compute the full covariance matrix of a dataset.

    Args:
    data (list of lists): A list of lists, where each inner list represents a sample and cont

    Returns:
```

```
    list of lists: The full covariance matrix of the p features.

    Raises:
    ValueError: If the input data is empty or if the inner lists have different lengths.
    """

    if not data:
        raise ValueError("Input data is empty")

    num_samples = len(data)
    num_features = len(data[0])

    if not all(len(row) == num_features for row in data):
        raise ValueError("Inner lists must have the same length")

    # Calculate the mean of each feature
    means = [sum(col) / num_samples for col in zip(*data)]

    # Calculate the covariance matrix
    cov_matrix = [[sum(((data[k][i] - means[i]) * (data[k][j] - means[j])) for k in range(num
                   for j in range(num_features)]
                  for i in range(num_features)]

    return cov_matrix
```
The time complexity of this solution is O(n*p^2), where n is the number of samples and p is t

=== GENERATION END ===

=== EVALUATION START ===
Status: NEEDS_IMPROVEMENT
Feedback:

The time complexity of O(n*p^2) is correct. However, there are some software engineering best

1. The function name `compute_covariance_matrix` is clear, but it would be better to use a mo

2. The argument `data` is a list of lists, which is not very descriptive. It would be better

3. The function does not check if the input data is a list of lists. It should add a check to

4. The function does not check if the number of samples is greater than 1. If the number of s

```
5. The function uses `zip(*data)` to transpose the data, which is not very efficient for larg

6. The function uses a list comprehension to calculate the covariance matrix, which can be ha

7. The function does not include a docstring example, which makes it harder for users to unde

8. The function does not handle the case where the number of samples is less than the number


=== EVALUATION END ===

Too many improvements needed. Stopping the process.

('\n```\ndef compute_covariance_matrix(data):\n    """\n    Compute the full covariance matri
 [{'thoughts': '\nBased on the task, I understand that I need to write a Python code to compu
   'result': '\n\nHere is the Python code to compute the full covariance matrix:\n```\ndef co
  {'thoughts': '',
   'result': '\n```\ndef compute_covariance_matrix(data):\n    """\n    Compute the full cova
```